AI ASSISTED CODING

Lab Assignment-21.1

Name : T.Shylasri

H.T.NO: 2303A51876

Batch-14(LAB-21)

Date: 02-04-2026

Lab 21 – Collaborative AI Coding: Pair Programming and Version Control Integration

Lab Objectives:

• Learn to use AI as a pair programmer for writing, reviewing, and optimizing code.

• Understand how to integrate AI-generated code into version control systems (Git).

• Practice collaborative workflows such as commit messages, branching, and merging.

Task 1 – AI-Assisted Pair Programming

• Use AI to generate an initial program (e.g., a simple To - Do List Manager).

• Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).

• Compare the original and improved versions to understand the role of AI in collaboration.

AI Prompt

#Generate a Python program for a To-Do List Manager that allows the user to add, view, mark as completed, and delete tasks using a menu-driven approach.

Prompt Screenshot

Initial Code

```python
#Generate a Python program for a To-Do List Manager that allows the user to add, view, mark as completed, and delete tasks using a menu-driven approach.
#Initial Code
tasks = []

while True:
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Mark Task as Completed")
    print("4. Delete Task")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
        task = input("Enter task: ")
        tasks.append(task)

    elif choice == '2':
        for i, task in enumerate(tasks):
            print(i + 1, task)

    elif choice == '3':
        index = int(input("Enter task number: ")) - 1
        tasks[index] = tasks[index] + " ✓"

    elif choice == '4':
        index = int(input("Enter task number: ")) - 1
        tasks.pop(index)

    elif choice == '5':
        break

    else:
        print("Invalid choice")
```

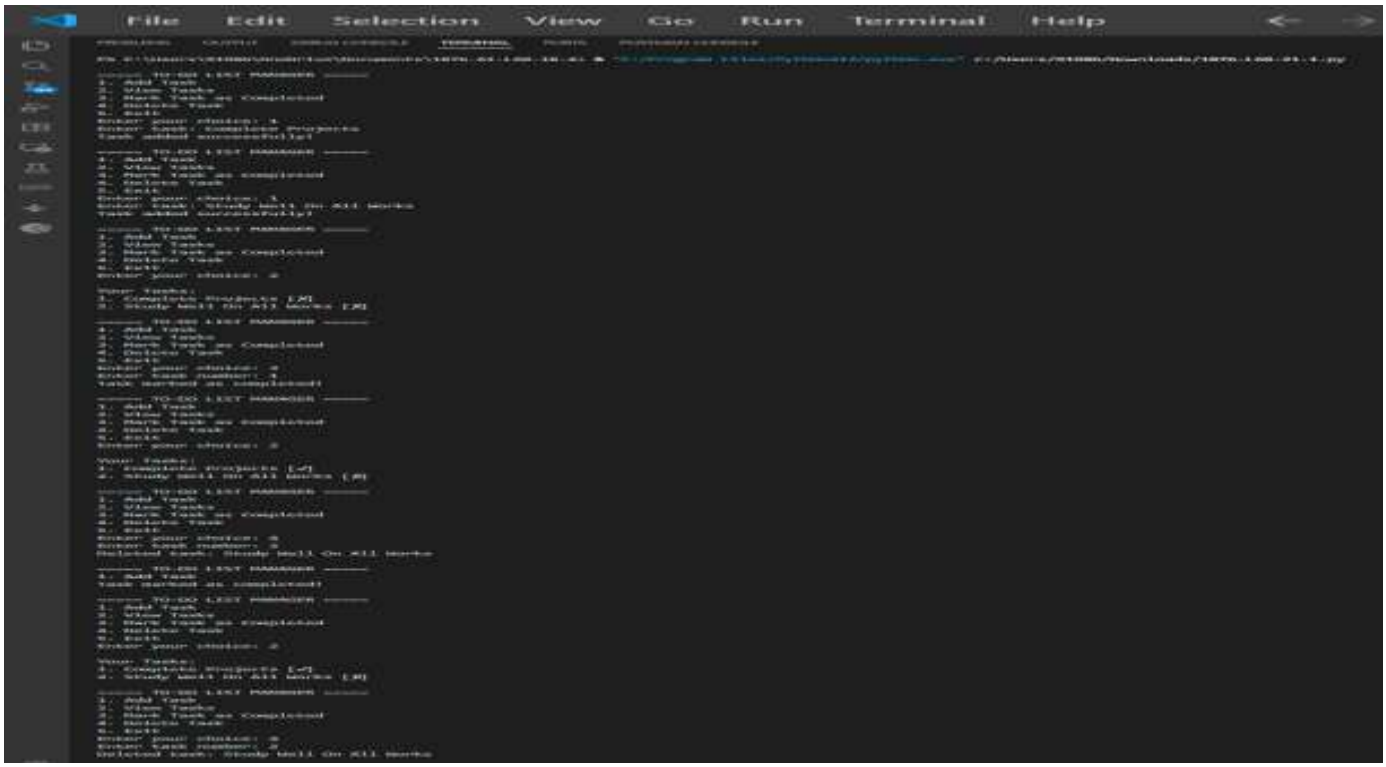
Output

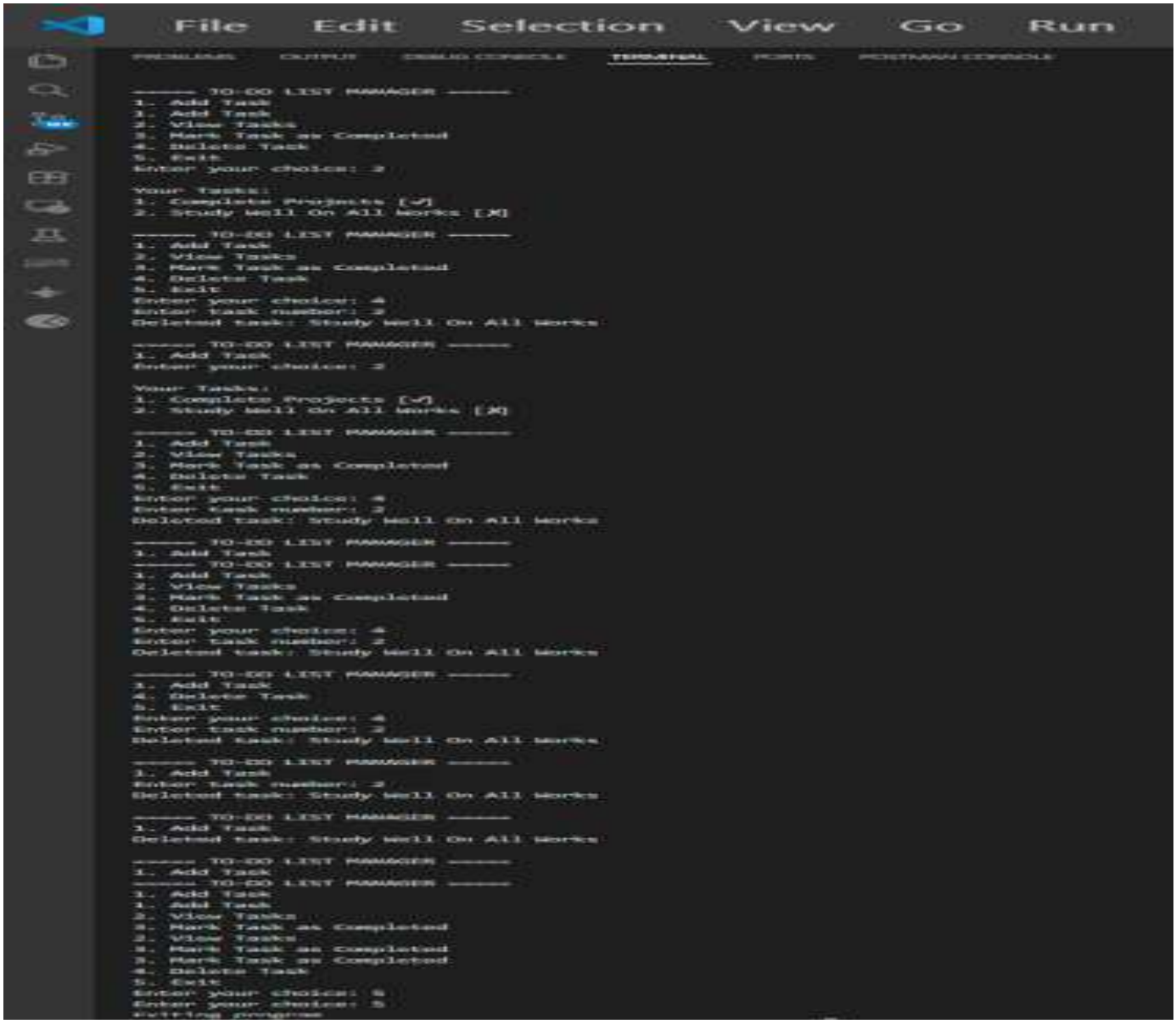
```
PS C:\Users\91986\OneDrive\Documents\1876-AI-LAB-18.4> & "C:/Program Files/Python313/python.exe" c:/Users/91986/Downloads/1876-LAB-21.1.py

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 1
Enter task: Complete Works

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 1
Enter task: Complete Projects

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 2
1 Complete Works
2 Complete Projects

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 3
Enter task number: 1

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 2
1 Complete Works ✓
2 Complete Projects

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 4
Enter task number: 2

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Exit
Enter your choice: 5
```

Improved Code

```python
#Generate a Python program for a To-Do List Manager that allows the user to add, view, mark as completed, and delete tasks using a menu-driven approach.
#Improved Code
def display_menu():
    print("\n------ TO-DO LIST MANAGER ------")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Mark Task as Completed")
    print("4. Delete Task")
    print("5. Exit")

def add_task(tasks):
    task = input("Enter task: ").strip()
    if task:
        tasks.append({"task": task, "done": False})
        print("Task added successfully!")
    else:
        print("Task cannot be empty.")

def view_tasks(tasks):
    if not tasks:
        print("No tasks available.")
        return
    print("\nYour Tasks:")
    for i, t in enumerate(tasks):
        status = "✓" if t["done"] else "✗"
        print(f"{i + 1}. {t['task']} [{status}]")

def mark_completed(tasks):
    try:
        index = int(input("Enter task number: ")) - 1
        if 0 <= index < len(tasks):
            tasks[index]["done"] = True
            print("Task marked as completed!")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")

def delete_task(tasks):
    try:
        index = int(input("Enter task number: ")) - 1
        if 0 <= index < len(tasks):
            removed = tasks.pop(index)
            print(f"Deleted task: {removed['task']}")
        else:
            print("Invalid task number.")
    except ValueError:
        print("Please enter a valid number.")

def main():
    tasks = []

    while True:
        display_menu()
        choice = input("Enter your choice: ")

        if choice == '1':
            add_task(tasks)
        elif choice == '2':
            view_tasks(tasks)
        elif choice == '3':
            mark_completed(tasks)
        elif choice == '4':
            delete_task(tasks)
        elif choice == '5':
            print("Exiting program...")
            break
        else:
            print("Invalid choice. Try again.")


if __name__ == "__main__":
    main()
```

Output





Observation

- AI generated a working base program quickly.
- The initial version lacked:
  - Input validation
  - Error handling
  - Clean structure
- The improved version:
  - Uses functions for modularity
  - Handles invalid inputs safely
  - Provides better user feedback

Explanation

AI acts as a **pair programmer** by:

- Writing initial code based on requirements
- Improving code through iterative prompts
- Suggesting best practices like modular programming
- Reducing development time

Justification

Using AI in programming:

- Enhances productivity
- Reduces human errors
- Helps beginners understand coding patterns
- Improves code readability and maintainability

The improved version is better because:

- It prevents crashes
- It is easier to extend
- It follows structured programming principles

Conclusion

AI-assisted pair programming is an effective method for:

- Rapid code generation
- Continuous improvement
- Learning coding standards

This task demonstrates how AI can:

- Act as a collaborator
- Improve code quality
- Support efficient software development

Task 2 – Version Control Integration

• Initialize a Git repository for your project.

• Perform the following steps:

1. Create and switch to a new branch.

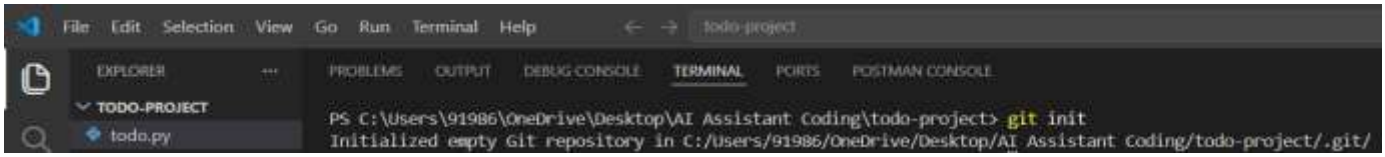
2. Add AI-assisted modifications.

3. Commit with a meaningful message.

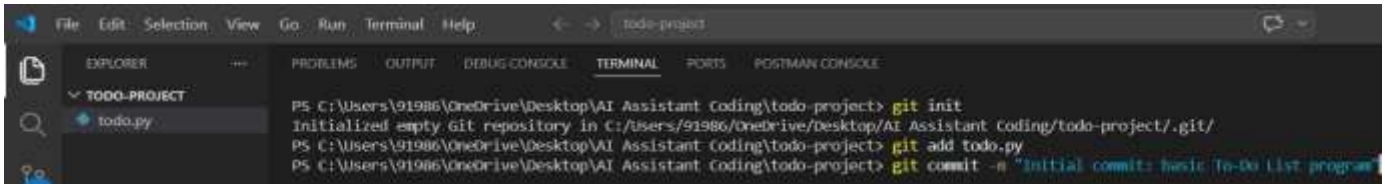
4. Merge changes back into the main branch.

• Reflect on how version control supports collaborative development with AI assistance.

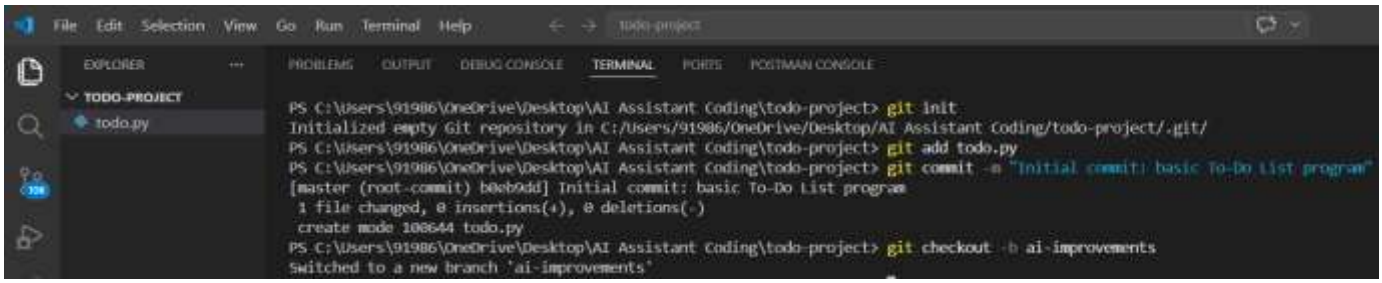
Initialize Git Repository



Add and Commit Initial Code



Create and Switch to New Branch



Modify Code Using AI

Merge Changes to Main Branch

```
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git checkout main
error: pathspec 'main' did not match any file(s) known to git
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git merge ai-improvements
Already up to date.
```

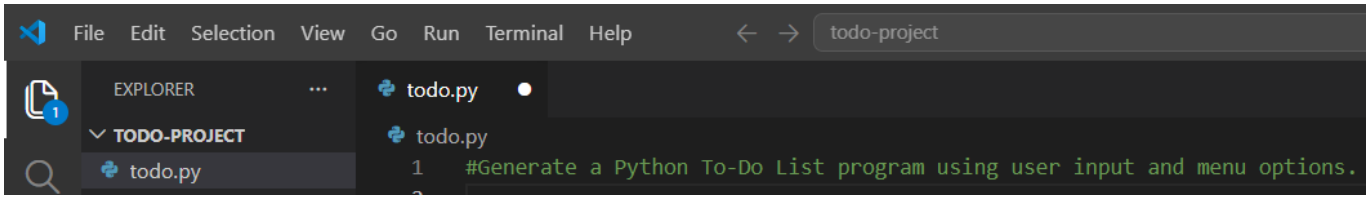
View Commit History

```
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git log --oneline
b0eb9dd (HEAD -> ai-improvements, master) Initial commit: basic To-Do List program
```

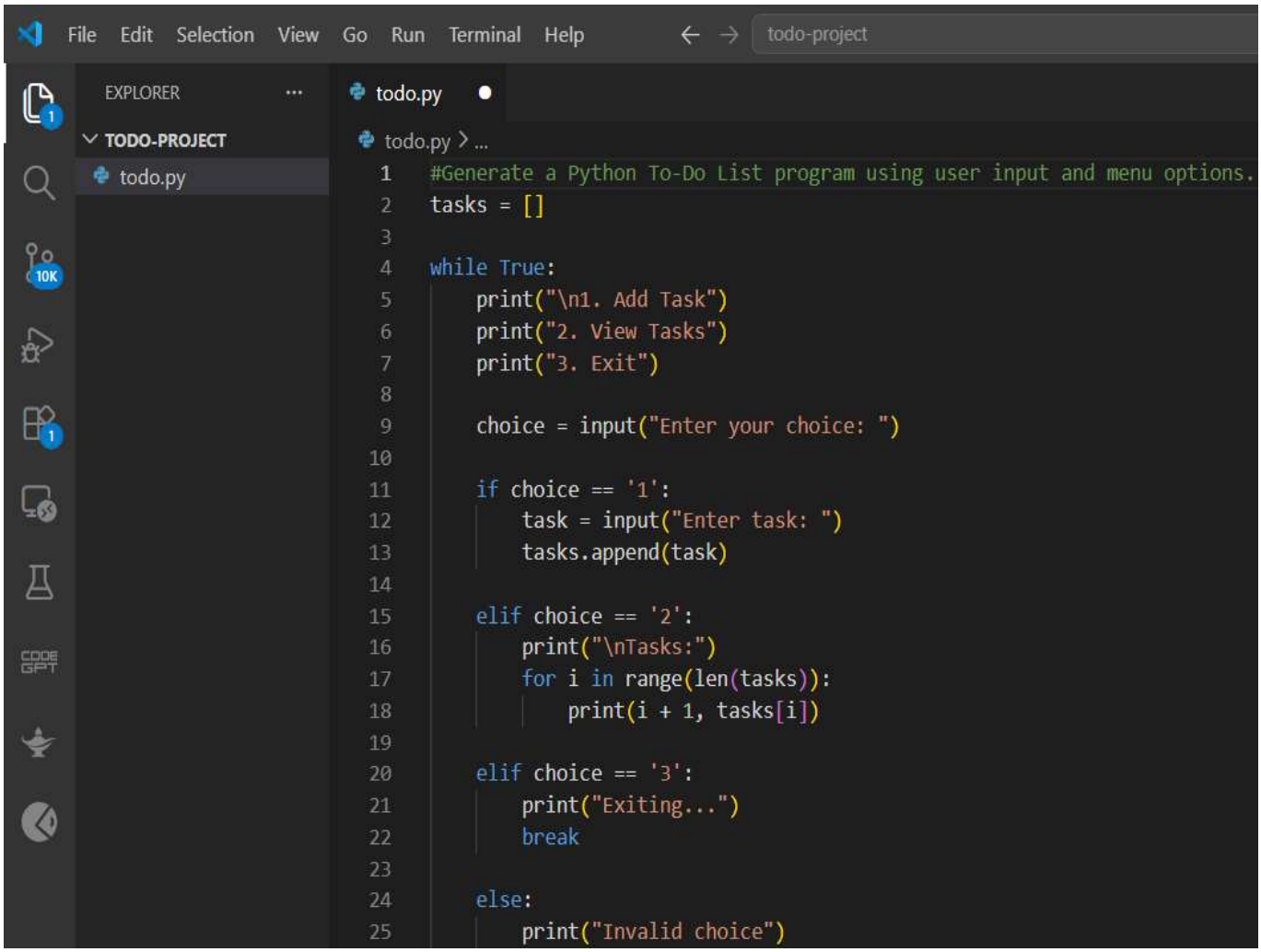
AI Initial Prompt

#Generate a Python To-Do List program using user input and menu options.

Prompt Screenshot



Initial Code



Output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/91986/OneDrive/Desktop/AI Assistant Coding/todo-project/todo.py"

1. Add Task
2. View Tasks
3. Exit
Enter your choice: 1
Enter task: 1

1. Add Task
2. View Tasks
3. Exit
Enter your choice: 2

Tasks:
1 1

1. Add Task
2. View Tasks
3. Exit
Enter your choice: 3
Exiting...
```

AI Improvement Prompt

#Improve the program using functions, error handling, and input validation.

Prompt Screenshot

```
File  Edit  Selection  View  Go  Run  Terminal  Help        todo-project

EXPLORER                todo.py
TODO-PROJECT            todo.py
todo.py
1   #Improve the program using functions, error handling, and input validation.
```

Improved Code

```
File  Edit  Selection  View  Go  Run  Terminal  Help        todo-project

EXPLORER                todo.py
TODO-PROJECT            todo.py > ...
todo.py
1   #Improve the program using functions, error handling, and input validation.
2   def add_task(tasks):
3       task = input("Enter task: ")
4       if task:
5           tasks.append(task)
6           print("Task added")
7
8   def view_tasks(tasks):
9       for i in range(len(tasks)):
10          print(i+1, tasks[i])
11
12  def main():
13      tasks = []
14      while True:
15          print("\n1.Add 2.View 3.Exit")
16          choice = input("Enter choice: ")
17
18          if choice == '1':
19              add_task(tasks)
20          elif choice == '2':
21              view_tasks(tasks)
22          elif choice == '3':
23              break
24          else:
25              print("Invalid")
26
27  main()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

1.Add 2.View 3.Exit
Enter choice: 1
Enter task: 2
Task added

1.Add 2.View 3.Exit
Enter choice: 3
```

Observation

- Git successfully tracks all code changes
- Branching allows testing AI improvements safely
- No data loss during modifications

Explanation

Git helps manage versions of code by:

- Saving different stages of development
- Allowing multiple versions using branches
- Combining updates through merging

Justification

Using Git with AI:

- Keeps original and improved code separate
- Helps test AI-generated changes safely
- Improves collaboration and organization

Conclusion

Version control with Git:

- Enhances development workflow
- Supports AI-assisted coding
- Ensures safe and efficient code management

Task 3 – AI for Commit Message Generation

- Use AI to generate concise, meaningful commit messages based on the changes made.
- Compare AI-generated messages with human-written ones.
- Discuss how AI can improve documentation and traceability in Git history.

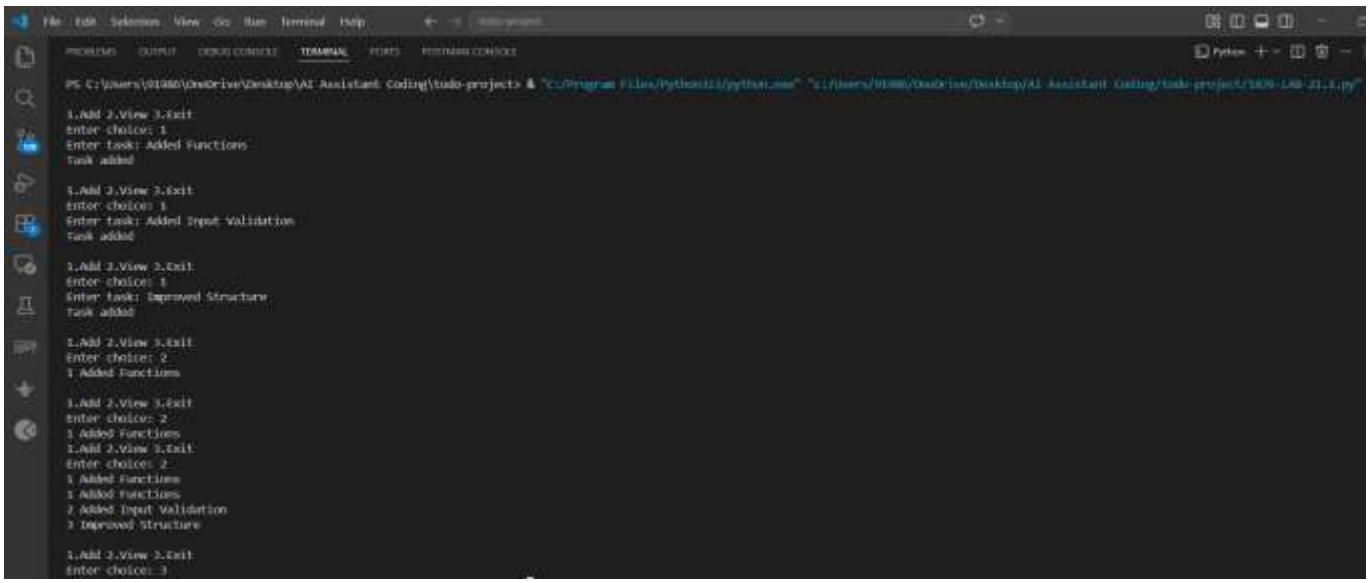
AI Prompt

#Generate a concise and meaningful Git commit message for a To-Do List program where functions, input validation, and error handling were added.

Prompt Screenshot

Code (Changes Made in Project)

Output



AI-Generated Commit Messages

Refactored To-Do app using functions and added input validation

Improved code structure with functions and basic error handling

Human-Written Commit Messages

Updated code

Made some changes

Observation

- AI-generated messages are clear and descriptive
- Human-written messages are often vague
- AI highlights important changes automatically

Explanation

AI analyzes the changes made in code and:

- Identifies key updates (functions, validation)
- Converts them into meaningful sentences
- Produces concise commit messages

Justification

Using AI for commit messages:

- Saves time for developers
- Ensures consistency in documentation
- Makes project history easy to understand

Conclusion

AI helps improve Git workflows by:

- Generating meaningful commit messages
- Enhancing documentation quality
- Supporting better collaboration and traceability

Task 4 – Pair Programming Simulation

• Student A writes the initial version of a program.

• Student B, with AI assistance, suggests improvements or adds features.

• Both students collaborate using Git branching and merging to integrate their work.

AI Prompt (Used by Student B)

#Improve the To-Do List program by adding functions, input validation, and better user interaction.

Prompt Screenshot

```
1876-LAB-21.1.py
1876-LAB-21.1.py
1  #Improve the To-Do List program by adding functions, input validation, and better user interaction.
```

Student A Code (Initial Version)

```
1876-LAB-21.1.py
1876-LAB-21.1.py > ...
#Improve the To-Do List program by adding functions, input validation, and better user interaction.
#Student A Code
tasks = []

while True:
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        task = input("Enter task: ")
        tasks.append(task)

    elif choice == '2':
        for i in range(len(tasks)):
            print(i+1, tasks[i])

    elif choice == '3':
        break
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

1. Add Task
2. View Tasks
3. Exit
Enter choice: 1
Enter task: Complete Projects

1. Add Task
2. View Tasks
3. Exit
Enter choice: 2
1 Complete Projects

1. Add Task
2. View Tasks
3. Exit
Enter choice: 3
```

Student B Code (Improved with AI)

```python
#Improve the To-Do List program by adding functions, input validation, and better user interaction.
#Student B Code
def add_task(tasks):
    task = input("Enter task: ")
    if task.strip() != "":
        tasks.append(task)
        print("Task added")
    else:
        print("Invalid task")

def view_tasks(tasks):
    if not tasks:
        print("No tasks available")
    else:
        for i in range(len(tasks)):
            print(i+1, tasks[i])

tasks = []

while True:
    print("\n1.Add 2.View 3.Exit")
    choice = input("Enter choice: ")

    if choice == '1':
        add_task(tasks)
    elif choice == '2':
        view_tasks(tasks)
    elif choice == '3':
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Output

```
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/91986/OneDrive/Desktop/AI Assistant Coding/todo-project/1876-LAB-21.1.py"

1.Add 2.View 3.Exit
Enter choice: 1
Enter task: Complete Tasks
Task added

1.Add 2.View 3.Exit
Enter choice: 2
1 Complete Tasks

1 Complete Tasks

1.Add 2.View 3.Exit
Enter choice: 3
Exiting...
```

Git Collaboration Steps

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git branch
  ai-improvements
* improvements
  master
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git add todo.py
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git commit -m "improved code using AI: added validation and functions"
On branch improvements
nothing to commit, working tree clean
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git status
On branch improvements
nothing to commit, working tree clean
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> git merge improvements
Already up to date.
```

Observation

- Student A created a basic working program
- Student B improved code using AI
- Git helped manage both versions safely

Explanation

Pair programming:

- One writes code (Student A)
- Other improves it (Student B with AI)
- Git helps combine both contributions

Justification

This approach:

- Improves code quality
- Encourages collaboration
- Uses AI effectively for enhancements

Conclusion

Pair programming with AI:

- Enhances productivity
- Improves code structure
- Makes teamwork efficient using Git

Task 5 – AI-Assisted Debugging

• Create a program with logical or syntax errors.

• Use AI to identify and fix the errors.

• Compare the buggy and corrected versions.

AI Prompt

#Find and fix errors in the following Python To-Do List program and provide the corrected code.

Prompt Screenshot

```
#Find and fix errors in the following Python To-Do List program and provide the corrected code.
```

Buggy Code (With Errors)

```
#Find and fix errors in the following Python To-Do List program and provide the corrected code.
#Buggy Code (With Errors)
tasks = []

while True:
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        task = input("Enter task: ")
        tasks.append(task)

    elif choice == '2':
        for i in range(len(tasks)):
            print(i+1 tasks[i])   # ❌ Missing comma

    elif choice == '3':
        print("Exiting...")
        break

    else
        print("Invalid choice")   # ❌ Missing colon
```

Output

```
PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:\Program Files\Python311\python.exe" "c:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project\todo.py"
  File "c:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project\todo.py", line 18
    print(i+1 tasks[i])   # ❌ Missing comma
          ^^^^^^^^^^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

Corrected Code (AI Fixed Version)

```
#Find and fix errors in the following Python To-Do List program and provide the corrected code.
#Corrected Code (AI Fixed Version)
tasks = []

while True:
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        task = input("Enter task: ")
        tasks.append(task)

    elif choice == '2':
        for i in range(len(tasks)):
            print(i+1, tasks[i])   # ✓ Fixed

    elif choice == '3':
        print("Exiting...")
        break

    else:
        print("Invalid choice")   # ✓ Fixed
```

Output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE                                   Python

PS C:\Users\91986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/91986/OneDrive/Desktop/AI Assistant Coding/todo-project/todo.py"

1. Add Task
2. View Tasks
3. Exit
Enter choice: 1
Enter task: Complete Assignments

1. Add Task
2. View Tasks
3. Exit
Enter choice: 2
1 Complete Assignments

1. Add Task
2. View Tasks
3. Exit
Enter choice: 3
Exiting...
```

Observation

- The buggy code produced syntax errors
- AI quickly identified missing symbols
- Corrected code runs without errors

Explanation

AI helps debugging by:

- Detecting syntax errors (missing comma, colon)
- Suggesting correct syntax
- Providing working version of code

Justification

Using AI for debugging:

- Saves time
- Reduces manual effort
- Helps beginners understand mistakes

Conclusion

AI-assisted debugging:

- Improves code quality
- Fixes errors quickly
- Enhances learning and productivity

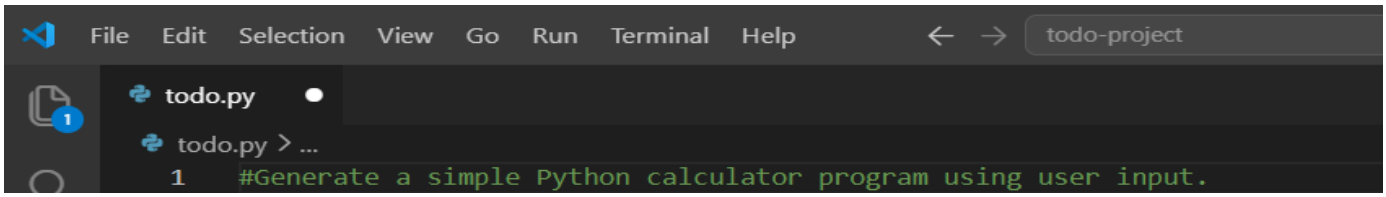
Task 6 – Feature Expansion using AI

- Develop a basic application (e.g., Calculator).
- Use AI to add advanced features (history, validation).
- Analyze improvements.

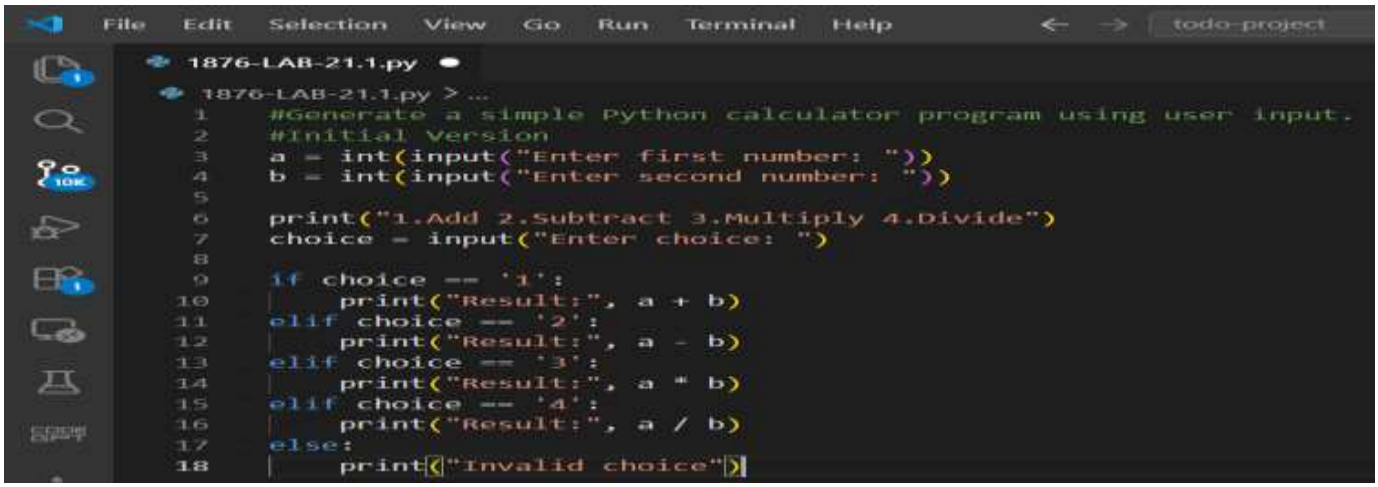
AI Initial Prompt

#Generate a simple Python calculator program using user input.

Prompt Screenshot



Initial Code

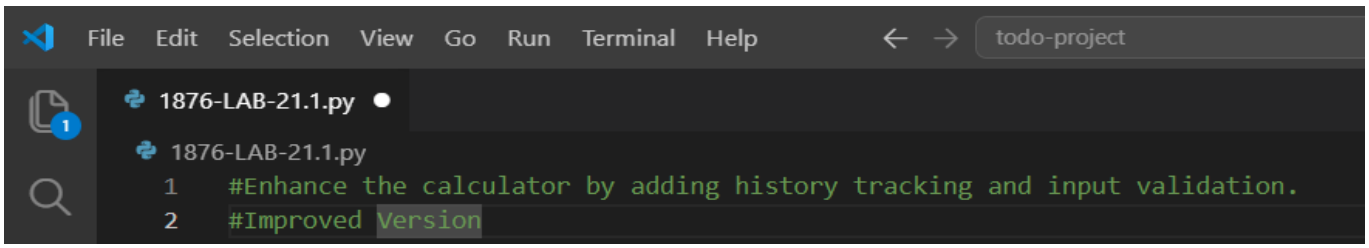


Output

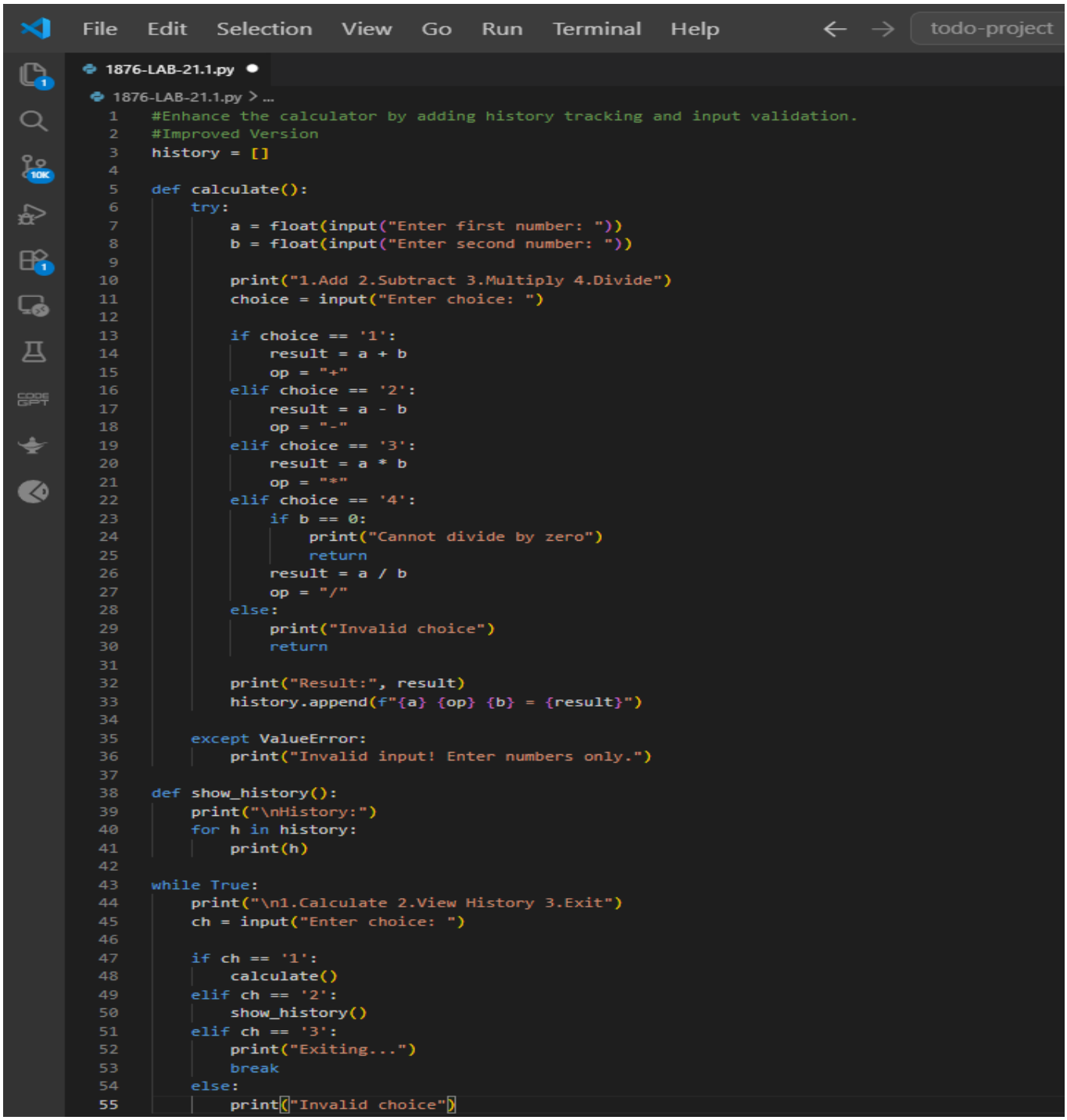
Improvement Prompt

#Enhance the calculator by adding history tracking and input validation.

Prompt Screenshot



Improved Code



Output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\01986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/01986/OneDrive/Desktop/AI Assistant Coding/todo-project/1876-LAB-21.
1.py"

1.Calculate 2.View History 3.Exit
Enter choice: 1
Enter first number: 2
Enter second number: 6
1.Add 2.Subtract 3.Multiply 4.Divide
Enter choice: 1
Result: 8.0

1.Calculate 2.View History 3.Exit
Enter choice: 2

History:
2.0 + 6.0 = 8.0

1.Calculate 2.View History 3.Exit
Enter choice: 3
Exiting...
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\01986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/01986/OneDrive/Desktop/AI Assistant Coding/todo-project/1876-LAB-21.1.py"

1.Calculate 2.View History 3.Exit
Enter choice: 1
Enter first number: 6
Enter second number: 10
1.Add 2.Subtract 3.Multiply 4.Divide
Enter choice: 2
Result: -4.0

1.Calculate 2.View History 3.Exit
Enter choice: 2

History:
6.0 - 10.0 = -4.0

1.Calculate 2.View History 3.Exit
Enter choice: 3
Exiting...
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\01986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/01986/OneDrive/Desktop/AI Assistant Coding/todo-project/1876-LAB-21.1.py"

1.Calculate 2.View History 3.Exit
Enter choice: 1
Enter first number: 6
Enter second number: 8
1.Add 2.Subtract 3.Multiply 4.Divide
Enter choice: 3
Result: 48.0

1.Calculate 2.View History 3.Exit
Enter choice: 2

History:
6.0 * 8.0 = 48.0

1.Calculate 2.View History 3.Exit
Enter choice: 3
Exiting...
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\01986\OneDrive\Desktop\AI Assistant Coding\todo-project> & "C:/Program Files/Python313/python.exe" "c:/Users/01986/OneDrive/Desktop/AI Assistant Coding/todo-project/1876-LAB-21.1.py"

1.Calculate 2.View History 3.Exit
Enter choice: 1
Enter first number: 40
Enter second number: 20
1.Add 2.Subtract 3.Multiply 4.Divide
Enter choice: 4
Result: 2.0

1.Calculate 2.View History 3.Exit
Enter choice: 2

History:
40.0 / 20.0 = 2.0

1.Calculate 2.View History 3.Exit
Enter choice: 3
Exiting...
```

Observation

- Basic version performs simple calculations only
- Improved version:
  - Stores history
  - Handles invalid input
  - Prevents division by zero

Explanation

AI enhances the program by:

- Adding new features (history)
- Improving user experience
- Handling errors effectively

Justification

Using AI for feature expansion:

- Reduces development time
- Improves functionality
- Makes programs more user-friendly

Conclusion

AI helps in:

- Expanding basic programs into advanced applications
- Improving reliability and usability
- Supporting efficient software development